EXECUTIVE REPORT · 13 NOVEMBER 2026

# AI-driven software delivery: specifications or governance?

Two ways to put AI agents to work on real software changes, what each one does, who sells it, and how to choose. Written for technology and business leaders in regulated organisations.

Summary 1 Why this matters 2 The market 3 Spec-driven toolkits 4 Governed platforms 5 Comparison 6 Reasoning 7 What to do Method and sources

**How to read the evidence.** Each fact carries a label. Read in source we read it in the tool's public source code on 13 November 2026. Published summary it comes from a published summary of a study or news report; we did not open the original. Vendor claim the vendor says so, and nobody independent has confirmed it. Reference design it is true of a governed delivery console we designed and prototyped; that prototype runs on demo data. Our judgement our own conclusion.

## Summary

**AI now writes code faster than organisations can review, approve and release it. The two approaches in this report attack that problem from opposite ends, and most large organisations will need both.**

1. **Spec-driven toolkits control what the agent is asked to build.** The team writes the requirement, plan and tasks before any code. The leading toolkits are free, open source and mature: GitHub Spec Kit is at version 1.4 and AWS AI-DLC at 2.11. Read in source
2. **Governed delivery platforms control who approves the work and what is released.** Every large delivery vendor shipped or announced one in 2026: GitHub, GitLab, Atlassian, ServiceNow, Harness, IBM and Thoughtworks. Vendor claim
3. **The two are converging.** Toolkits are adding approval gates and audit trails. Platforms are adding planning and specification agents. Read in source Vendor claim

4. **Neither has independent proof of better outcomes yet.** The largest study of written specifications found no effect on defects. Platform results are vendor-reported. Published summary

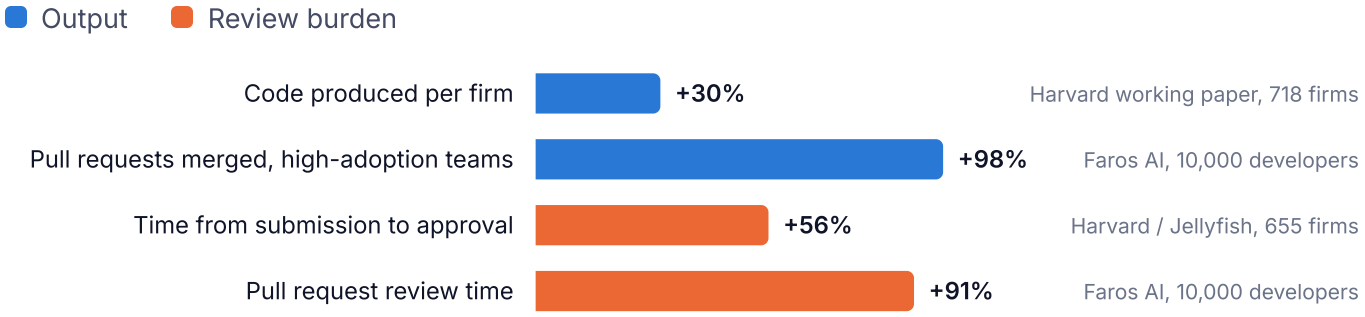
5. **Our advice: adopt a toolkit now, put governance in the system of record you already own, and build custom only for what vendors do not cover.** Today that gap is the experience for business approvers and one evidence record across tools. Our judgement

SECTION 1

## Why this matters: the constraint has moved

For two years the question was whether AI could write useful code. That is settled. The new constraint is everything that happens after the code is written.

### More code is produced, and review takes longer




Change associated with AI adoption. Published summary Two versions of the Harvard paper report 49% and 56% for approval time; we show the later one.

### Developers use AI but do not trust it with decisions




Stack Overflow Developer Survey 2026. Published summary

### When review cannot keep up, it gets skipped

**48%** of merged changes were reviewed before a company-wide AI mandate

**21%** were reviewed afterwards

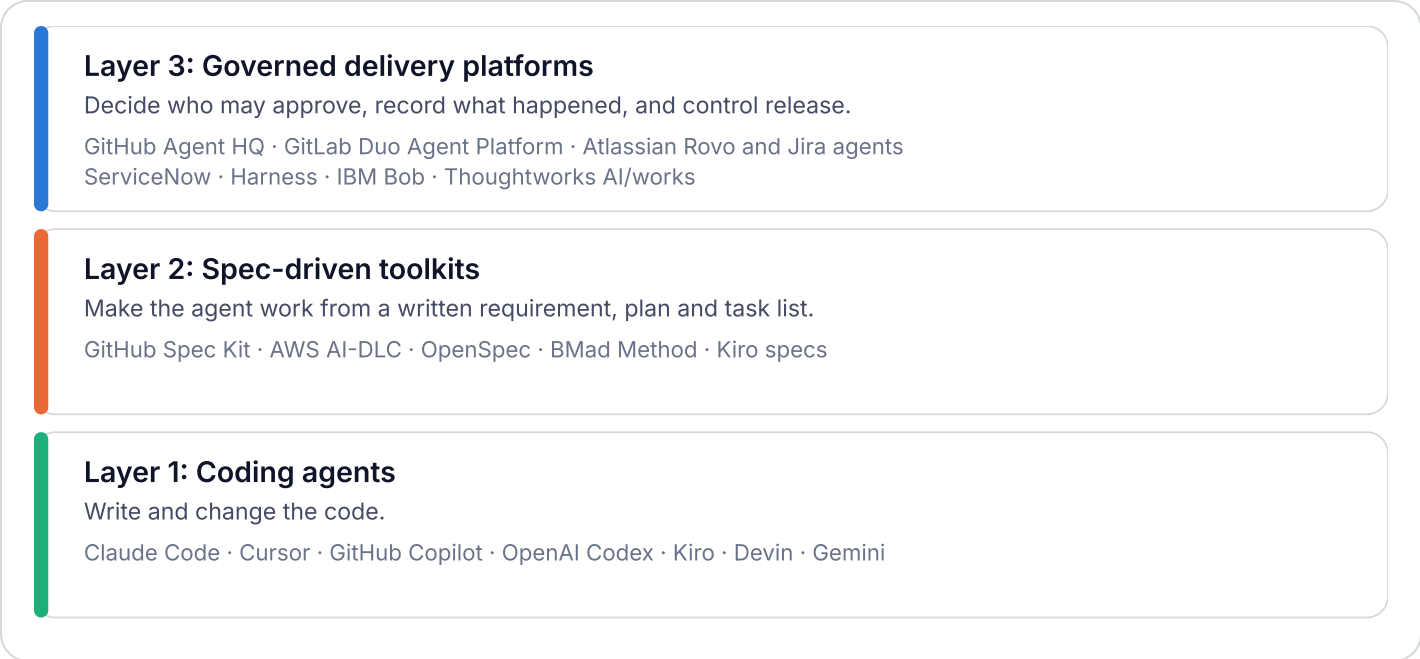
One enterprise, studied over time (arXiv 2607.01904). Published summary

- **Delivery stability falls as AI use rises.** Google's DORA research links AI adoption to higher throughput and also to more instability. Published summary
- **Returns are not yet visible.** McKinsey's 2026 survey reports that 80% of workers see productivity gains, but only 6% of organisations attribute significant earnings to AI. Published summary
- **So the value is no longer in generating code.** It is in making sure the right thing is built, checked and released under control. That is what both approaches in this report try to do. Our judgement

SECTION 2

## The market in three layers

AI-driven software delivery is easier to understand as three layers. Each layer depends on the one below it. This report compares layers 2 and 3.




- **Layer 1 is a settled, crowded market.** Gartner's first Magic Quadrant for Enterprise AI Coding Agents (May 2026) names Anthropic, Cursor, GitHub and OpenAI as Leaders, with AWS, Google, Alibaba Cloud and Cognition as Challengers. Published summary
- **Layer 2 is open source and moving fast.** All four toolkits we read had commits in the two days before this report. Read in source
- **Layer 3 is where the large vendors are now competing.** Each one extends the system of record it already owns: repository, tracker, service desk or pipeline. Our judgement

SECTION 3

# Approach A: spec-driven toolkits

In spec-driven development the team writes down what to build and why before the agent writes code. The requirement, the plan and the task list are text files kept in the repository beside the code. The agent is told to work from those files and nothing else.

## The leading toolkits

| Toolkit | Owner and licence | Version read | What stands out |
|---|---|---|---|
| **GitHub Spec Kit** | GitHub; MIT; free | 1.4.0, 10 Nov 2026 | Drives 43 coding agents. 198 community extensions and 42 presets. Workflows with human review gates. Private catalogs for approved add-ons |
| **AWS AI-DLC** | AWS Labs; MIT-0; free | 2.11.4, 13 Nov 2026 | 5 phases and 33 stages from idea to operation. 14 agents. 11 workflow profiles, from "Express" (7 stages) to "Enterprise" (30). Approval gates and a 134-event audit trail |
| **OpenSpec** | Fission AI; MIT; free | 2.0.0, 11 Nov 2026 | Lightweight. Describes each change as a proposal against existing behaviour, so it suits existing systems |
| **BMad Method** | Community; MIT; free | 6.12 stable, 12 Nov 2026 | Agile framing with role-based agents. Sizes the process to the change |
| **Kiro specs** | AWS; commercial | Not read | Built into the Kiro editor. Requirements, design and tasks, each approved by the developer. Enterprise sign-on and policy controls |

Rows 1 to 4 Read in source. Kiro Published summary.

## How a change moves through Spec Kit

| Step | What the developer runs | What it produces |
|---|---|---|
| Once | `constitution` | The project's standing rules |
| 1 | `specify`, then optional `clarify` | The specification: what and why |
| 2 | `plan` | The technical approach |
| 3 | `tasks`, then optional `analyze` | The task list, and a report of inconsistencies |
| 4 | `implement` | Code and tests |
| 5 | `converge` | A check of the code against the specification. Unbuilt work is added as new tasks |

**Read in source** It also ships separate, shorter processes for bug fixing and for assessing an idea.

### What it does well

- **Free and available now.** No procurement.
- **No lock-in.** Works with almost any coding agent.
- **Fits how developers work.** Everything is in the repository and versioned by Git.
- **Adaptable.** Presets, extensions and workflows let a team shape the process.
- **Community add-ons fill gaps.** Multi-repository dispatch, Azure DevOps with human sign-off, Jira and Linear sync all exist.

Read in source

### Where it falls short

- **Built for developers only.** No screen for a product owner, release manager or auditor.
- **Approval has no identity in the core.** A gate is a prompt in a terminal. AWS AI-DLC's gates are "not assigned to any specific role", one reviewer notes.
- **Documents drift from code.** Spec Kit's core has "no dedicated command" to reconcile them.
- **Add-ons are unvetted.** Maintainers "do not review, audit, endorse, or support" community code.
- **Stops before release.** Deployment and rollback are left to other tools.
- **Heavy for small changes.** Thoughtworks' Technology Radar reports "verbose markdown" and "instruction bloat", and rates Spec Kit "Assess", not "Adopt".

Read in source for the quoted Spec Kit text; Published summary for the reviewer and Thoughtworks.

## Does it improve outcomes?

| Study | Finding | Weight to give it |
|---|---|---|
| Hill, ESEM 2026. 88,052 pull requests, 119 open-source projects | Specification quality predicted neither defects (p = 0.164) nor rework (p = 0.860) | Peer reviewed. But it measured ordinary specifications, mostly links to issues. No project in the sample used a spec-driven toolkit |

| | | |
|---|---|---|
| Uvik benchmark, 2026. 50 tickets | 0.46 defects per merged ticket with a spec workflow, 0.86 without | Vendor-produced and small |
| Farrag, arXiv 2605.01160 | Argues that specification discipline, not model capability, is the binding constraint on dependable AI-written software | A review and a three-team pilot, not a controlled trial |
| OrangeLoops case study, 2026 | Code generation was much faster. Stabilising, integration and review still took four days | One project |

All **Published summary**. No independent study has yet isolated the effect of a spec-driven toolkit on defects or delivery time.

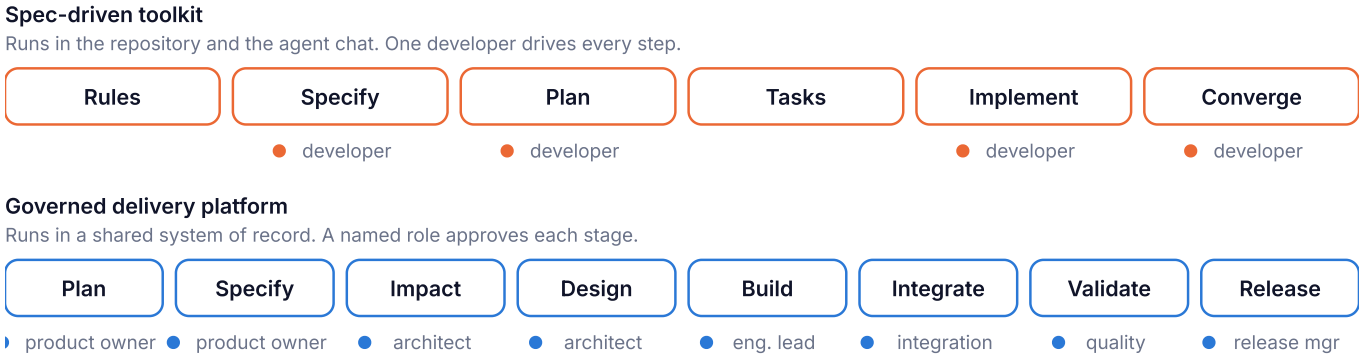
SECTION 4

# Approach B: governed delivery platforms

A governed delivery platform treats an AI agent like a new member of staff inside an existing control framework. The agent may do the work, but a named person approves each important step, the approval is recorded, and release is controlled by policy.

## A reference design

To make the category concrete we use a governed delivery console that we designed and prototyped. It is a design, not a product. It shows the full pattern in one place.




Dots show who approves. Read in source for the toolkit flow; Reference design for the platform flow.

| Element | In the reference design |
|---|---|
| Stages | Eight, from plan to release, with a named approver and a backup for each |
| Roles | Nine roles against seventeen permissions. A requester cannot approve their own plan |
| Evidence | Every document is versioned and fingerprinted. One audit log covers decisions, access and permission changes |
| Explanation | Each approval screen shows how the agent reached its output: sources, checks, confidence and open doubts |
| Build | Handed to a separate coding agent in the developer's editor |
| Release | Scheduling, freeze windows, hold with a recorded reason, and rollback |

| | |
|---|---|
| Limits | A prototype on demo data. One fixed workflow for every size of change. A manual hand-off to the coding agent. One tool ecosystem |

Reference design

## Enterprise products that follow this pattern

| Vendor and product | Status | How it governs agents | Starts from |
|---|---|---|---|
| **GitHub** Agent HQ, with Copilot coding agent | Announced late 2025. Governance controls reported in public preview | One "mission control" for agents from Anthropic, OpenAI, Google, Cognition and xAI. Admin policies, audit logging, agent identity, limits on which branches agents may change | The repository |
| **Microsoft** Azure Boards with Copilot coding agent | Generally available | A work item is assigned to the agent, which opens a pull request. Custom agents and model selection. Microsoft advises moving repositories to GitHub for the full experience | The work tracker |
| **GitLab** Duo Agent Platform | Generally available since 15 Jan 2026 | Planner and Security Analyst agents, plus external agents such as Claude Code and Codex. Flows run as pipeline jobs under an identity that never exceeds the triggering person's access | The repository and pipeline |
| **Atlassian** Rovo Studio, agents in Jira, Rovo Dev | Generally available, 2026 | Agents are assigned work items and act within Jira's existing permissions and approval workflows. Roles, approvals, versioning and audit controls. Agent changes land in the same history as human work | The work tracker |

| | | | |
|---|---|---|---|
| **ServiceNow** Build Agent, App Engine Management Center, AI Control Tower | Build Agent generally available. Expanded Control Tower due August 2026 | Deployment approvals, release management and lifecycle governance. Control Tower discovers, observes and can stop any agent acting outside its permissions | The service desk and change record |
| **Harness** Autonomous Worker Agents, Agent DLC | Launched mid-2026 | The same policy-as-code, access roles, approvals and change windows that gate human deployments also gate agents. Approvals can be triggered by risk, not applied to everything | The delivery pipeline |
| **IBM** Bob | Generally available since 28 Apr 2026 | Role-based agents across planning, design, coding, testing and deployment. Policy enforcement and human-in-the-loop controls. IBM reports 80,000 internal users | The developer's editor |
| **Thoughtworks** AI/works on Agent/works | Launched Jan 2026. Version 2.0 in Aug 2026 | Legacy analysis, requirements, specification generation, code and tests in one platform. A "Knowledge Fabric" of enterprise context. Runs on a governed agent control plane | A consulting engagement |
| **AWS** Kiro autonomous agent and frontier agents | Preview | Agents work in sandboxes and open pull requests for people to review. Also publishes the open-source AI-DLC method described in Section 3 | The cloud account |
| **Cognition** Devin Enterprise | Generally available | Single sign-on, audit logs, private deployment. Named users include Goldman Sachs and Citi | The coding agent |

Mostly Vendor claim, taken from announcements and trade coverage Published summary. We have not tested any of these products. Features in preview may have changed.

- **Every vendor starts from the record it already owns.** None of them offers one approval record across the tracker, the repository, the pipeline and the service desk. Our judgement
- **Most are still built for technical users.** Atlassian and ServiceNow are the exceptions, because their existing users include business staff. Our judgement
- **Banks are already using them.** Reported developer productivity gains are modest: 5 to 15% at Citi and 10 to 20% at JPMorgan. The Evident AI Index reports that only 1% of bank AI use cases disclosed a financial return in 2026. Published summary

SECTION 5

# Comparison

We scored each option on twelve capabilities, from 0 (absent) to 3 (strong and built in). The scores are our reading of public source code, documentation and announcements. They are not test results.

## Capability matrix

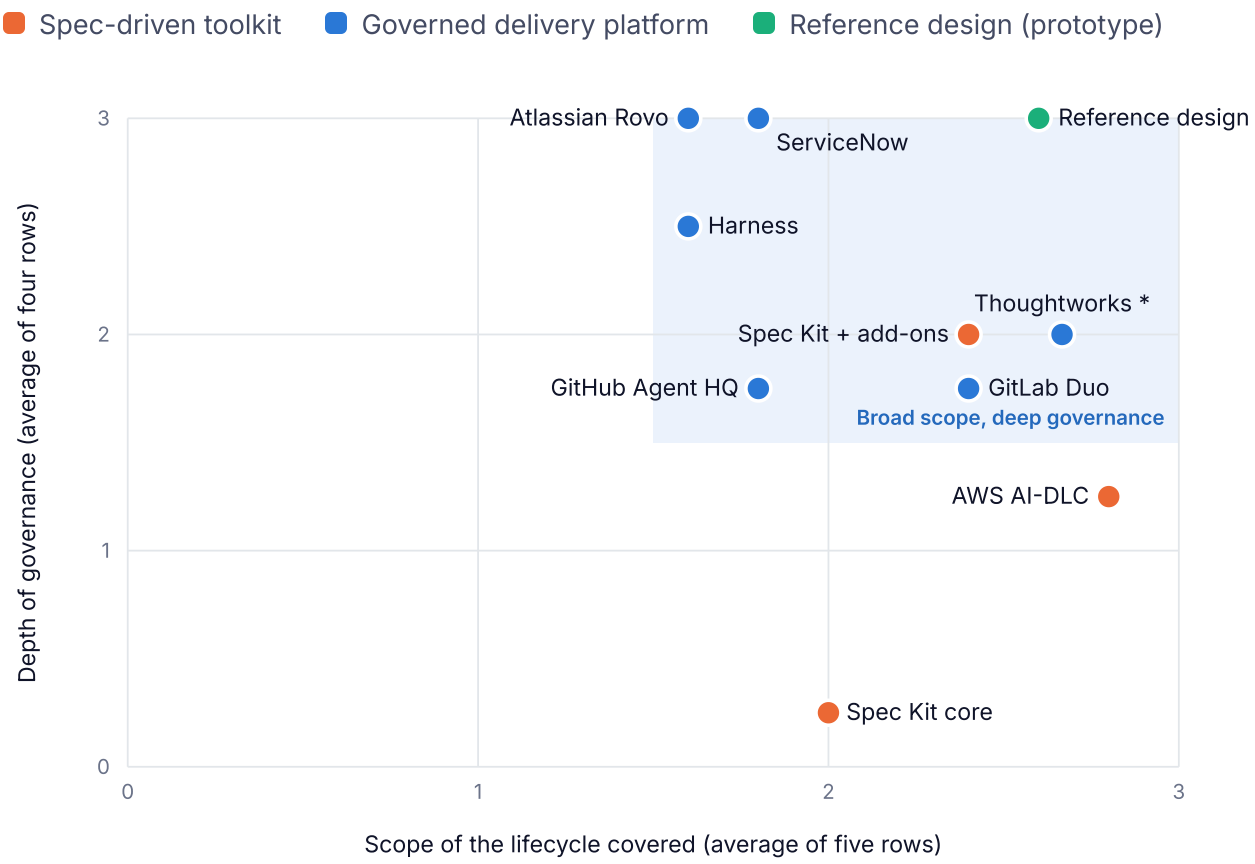
0 absent · 1 partial, or through an add-on · 2 present · 3 strong and built in

? not enough public evidence

| | Spec Kit core | Spec Kit + add-ons | AWS AI-DLC | GitHub Agent HQ | GitLab Duo | Atlassian Rovo | Service-Now | Harness | Thought-works | Reference design |
|---|---|---|---|---|---|---|---|---|---|---|
| **Scope of the lifecycle** | | | | | | | | | | |
| Written requirement before code | 3 | 3 | 3 | 1 | 2 | 2 | 1 | 0 | 3 | 3 |
| Design and impact analysis | 2 | 2 | 3 | 1 | 1 | 1 | 1 | 1 | ? | 3 |
| Code written by an agent | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 1 | 3 | 2 |
| Verification and test gates | 2 | 3 | 3 | 2 | 3 | 1 | 2 | 3 | 2 | 2 |
| Release, deploy and rollback | 0 | 1 | 2 | 2 | 3 | 1 | 3 | 3 | ? | 3 |
| **Depth of governance** | | | | | | | | | | |
| Human approval gates | 1 | 3 | 3 | 2 | 2 | 3 | 3 | 3 | ? | 3 |
| Named roles, separation of duties | 0 | 2 | 0 | 2 | 2 | 3 | 3 | 3 | ? | 3 |
| Central audit trail | 0 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 2 | 3 |
| Usable by non-developers | 0 | 1 | 0 | 1 | 1 | 3 | 3 | 1 | ? | 3 |
| **Practicalities** | | | | | | | | | | |
| Work across several repositories | 0 | 2 | ? | 2 | 2 | 2 | 1 | 2 | ? | 2 |
| Choice of agent and model | 3 | 3 | 3 | 3 | 3 | 2 | 2 | 2 | ? | 1 |
| In production use today | 3 | 2 | 3 | 2 | 3 | 3 | 2 | 3 | 2 | 0 |

**Our judgement** "Spec Kit + add-ons" includes the community multi-repository and Azure DevOps packages. "Reference design" is a prototype, which is why it scores 0 for production use.

## Where each option sits




**Our judgement** Each point is the average of the matrix rows above. Unknown cells are left out. * Thoughtworks is placed on four of nine rows only.

## What the charts show

- **Toolkits score high on requirements and code, and low on governance.** Spec Kit's core scores 0 on roles, audit and non-developer access.
- **Add-ons move toolkits a long way up.** With community packages, Spec Kit reaches the level of the weaker platforms on governance. Those packages are single-author and unvetted.
- **Platforms score high on governance and release, and lower on requirements.** Most start work at a ticket, not at a reviewed specification.
- **No shipping product reaches the top-right corner on every row.** The reference design does on paper, but it is not built.

**Our judgement**

## Side by side

| Question | Spec-driven toolkit | Governed delivery platform |
|---|---|---|
| What does it control? | The input: what the agent is asked to build | The process: who approves and what is released |
| Who uses it? | Developers | Developers, plus approvers, release and risk staff |
| Where does the record live? | Text files in the repository | A shared system of record |
| What does approval mean? | The developer moves to the next step | A named role signs off, and it is logged |
| Cost to start | None. Open source | Licence, integration and change management |
| Time to start | Hours | Months |
| Main risk | Documents nobody outside engineering reads; unvetted add-ons | Slow, heavy process; dependence on one vendor |
| Evidence of benefit | Weak and mixed | Vendor-reported only |

**Our judgement**, summarising Sections 3 and 4.

SECTION 6

# Reasoning: how to think about the choice

1. **They are different kinds of control, so they do not substitute for each other.** A specification is a preventive control: it reduces the chance the wrong thing is built. An approval record is an authorising control: it proves a responsible person accepted the result. Auditors and regulators ask for the second. Engineers benefit most from the first.
2. **The bottleneck is review, and only one approach addresses it directly.** Toolkits add documents to review. Platforms can route, batch and risk-tier approvals. Harness, for example, triggers approval by policy and impact. A platform that applies every gate to every change will make the bottleneck worse.
3. **The layers are merging.** AWS AI-DLC already has gates, an audit trail and an "Enterprise" profile. A Spec Kit add-on has named sign-offs tied to a fingerprint of the signed files. From the other side, GitLab and Thoughtworks have planning and specification agents. Within one or two years the large vendors are likely to offer both layers together.
4. **That changes the case for building your own.** A custom governed platform now competes with the roadmaps of GitHub, GitLab, Atlassian and ServiceNow. It is worth building only where they are weak: a clear experience for business approvers, one evidence record across several tools, and explanations of what the agent did.
5. **Regulation raises the floor but has not set a deadline for most of this.** The European Union has postponed high-risk AI obligations to December 2027. Existing rules on change management and separation of duties already apply to AI-written changes, whoever wrote them.

Our judgement, drawing on Sections 1 to 5. The EU date is Published summary.

## Which to choose

| If your situation is | Start with | Why |
|---|---|---|
| Engineering teams already use coding agents with little structure | A spec-driven toolkit | Free, fast, and improves the quality of what agents are asked to do |

| | | |
|---|---|---|
| You must show auditors who approved each change | Governance in your existing system of record | Your tracker or pipeline vendor now offers agent controls, and your approvers already use it |
| Changes cross many teams, repositories and tools | Both layers, connected | A toolkit per repository, with one platform holding approvals and release |
| Business owners must approve in plain language | A platform with strong non-developer access, or a custom front end | Toolkits and most platforms assume a technical user |
| Most changes are small and low risk | A light toolkit profile and risk-based approvals | Full process on every change will be bypassed |

Our judgement

SECTION 7

# What to do

1. **Adopt a spec-driven toolkit in engineering now.** Start with one team and one toolkit. Use a light profile for small changes.
2. **Turn on the agent governance your current vendors already ship.** Ask each one for agent identity, audit logging, approval policy and a stop control before buying anything new.
3. **Build custom only for the gap.** Today that is the business approver's experience and a single evidence record across tools. Connect to toolkits and platforms; do not replace them.
4. **Measure review time and failed changes, not lines of code.** Take a baseline before the pilot. Without one, no claim of benefit can be tested.
5. **Tier approvals by risk from the first day.** Reserve full sign-off for changes that need it.
6. **Treat community add-ons as third-party software.** Review the code, pin the version, and host approved packages in a private catalog.

### A ninety-day plan

| Days | Action | Result |
|---|---|---|
| 1 to 30 | Record a baseline for one team: review time, lead time, failed changes. Choose one toolkit and one small set of approved add-ons | A baseline and an approved setup |
| 31 to 60 | Run real changes through the toolkit. Switch on agent audit and approval policy in the existing tracker or pipeline | Twenty or more changes delivered with a record of who approved |
| 61 to 90 | Compare against the baseline. List what approvers and auditors still could not see or do | A decision on what, if anything, to build |

Our judgement

## Method, limits and sources

- **Source reading.** We cloned seven public repositories on 13 November 2026 and read their documentation, catalogs and templates. We did not install or run them, so behaviour is as documented, not as observed.
- **Published summaries.** Studies, surveys and product news come from search summaries of the named sources. We could not open the original pages. Check any figure before quoting it outside your organisation.
- **Scores.** The matrix and positioning chart are our judgement. Two analysts could reasonably differ by one point on any cell. Cells marked "?" had too little public evidence to score.
- **Not covered.** Pricing, security certification, and hands-on product testing. Google's agent products are omitted because reports about their status conflict.
- **Conflicts we noticed.** Sources disagree on whether Gartner named three or four Leaders, on Kiro's general-availability date, and on the Harvard paper's approval-time figure.

### Read in source

- github/spec-kit (commit of 12 Nov 2026)
- awslabs/aidlc-workflows (13 Nov 2026)
- Fission-AI/OpenSpec · bmad-code-org/BMAD-METHOD

- Spec Kit add-ons: multi-repo, multi-repo-branching, Azure DevOps

### Research and surveys

- Hill, "Specification Artifacts in Open-Source Pull Request Workflows Do Not Reduce Defects", ESEM 2026
- Farrag, "The Productivity-Reliability Paradox", arXiv 2605.01160
- "AI Writes Faster Than Humans Can Review", arXiv 2607.01904
- Harvard working paper on coding agents and code review (coverage)
- DORA reports on AI-assisted development (InfoQ)
- Stack Overflow Developer Survey 2026
- McKinsey, "The AI revolution in software development"
- Gartner Magic Quadrant for Enterprise AI Coding Agents, 2026 (coverage)
- Thoughtworks Technology Radar: GitHub Spec Kit
- Böckeler, "Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl"
- Uvik benchmark · OrangeLoops case study

### Vendors and market

- GitHub Agent HQ (VentureBeat) · Azure Boards and Copilot coding agent
- GitLab Duo Agent Platform general availability
- Atlassian Team '26: Rovo · governed agent loops
- ServiceNow Build Agent · AI Control Tower (The New Stack)
- Harness Agent DLC · policy as code
- IBM Bob · The Register's coverage
- Thoughtworks AI/works 2.0 · Agent/works
- AWS AI-DLC explained · Cognition (Sacra)
- Goldman Sachs and agentic engineering (Forbes)
- EU AI Act: postponed high-risk deadlines (Gibson Dunn)